This CVPR Workshop paper is the Open Access version, provided by the Computer Vision Foundation. Except for this watermark, it is identical to the accepted version; the final published version of the proceedings is available on IEEE Xplore.

Enhancing LLM-Based Neural Network Generation: Few-Shot Prompting and Efficient Validation for Automated Architecture Design

Raghuvir Duvvuri*, Chandini Vysyaraju, Avi Goyal, Dmitry Ignatov†, Radu Timofte
Computer Vision Lab, CAIDAS & IFI, University of Würzburg, Germany
Corresponding authors: *dvenkataraghuvir@gmail.com, †dmytro.ignatov@uni-wuerzburg.de

Abstract

While large language models (LLMs) have unlocked a new paradigm for automated neural architecture design, their generative power remains poorly understood: the critical question of how in-context example count governs generation quality has never been systematically investigated. Building on the NNGPT[1]/LEMUR[2] ecosystem, we conduct the first empirical study of few-shot prompting for neural architecture generation. On complex fine-grained tasks, where few-shot context matters most, three supporting examples (n=3) yield statistically significant gains over single-example prompting (CIFAR-100: $+11.6\%$, p=0.001, Cohen's d=0.73; CelebA-Gender: $+6.5\%$, p=0.038), while also achieving the highest dataset-balanced mean accuracy of 53.1% across six benchmarks. We generate and evaluate 1,900 neural architectures produced by LLMs, rigorously assessing how the number of supporting examples ($n \in \{1, \ldots, 6\}$) shapes generation stability, architectural diversity, and early-epoch validation performance. Larger context sizes (n>3) lead to statistically significant degradation on more structured benchmarks (ImageNette: -14.5%, p=0.010; CIFAR-10: -8.4%, p=0.016), and at n=6, generation collapses entirely (99.8% failure rate), establishing a hard empirical upper bound on prompt context. Qualitative analysis reveals that n=3 uniquely enables architectural pattern synthesis (hybrid ResNet-DPN and ResNet-AlexNet structures) absent under single-example prompting. To support scalable deployment, we introduce whitespace-normalized hashing for real-time duplicate detection, achieving a $100\times$ speedup over AST-based methods. Critically, this sub-millisecond deduplication primitive is what makes the 1,900-architecture scale of this study computationally feasible, eliminating redundant GPU training at negligible overhead. Together, these findings establish prompt context calibration and lightweight validation as key principles for scalable, resource-efficient LLM-driven architecture design.

[1] https://github.com/ABrain-One/NN-GPT
[2] https://github.com/ABrain-One/NN-Dataset

1. Introduction

The design of effective neural network architectures traditionally relies on domain expertise and iterative experimentation. Neural Architecture Search (NAS) [26, 33] methods automate this process, but often require substantial computational resources and carefully engineered search spaces. Recently, Large Language Models (LLMs) have emerged as a promising alternative paradigm, leveraging code generation capabilities to synthesize neural architectures directly from natural language prompts [5, 30]. This approach shifts architecture design from gradient-based search toward generative modeling of network structure.

The Promise and Gap of LLM-Based Architecture Search. Recent advances have demonstrated that LLMs can serve as architecture generators through code synthesis. EvoPrompting [3] combines evolutionary algorithms with prompt-tuned LLMs, achieving competitive results on MNIST-1D and CLRS benchmarks. LLMatic [22] applies quality-diversity search with LLMs to generate diverse architectures on CIFAR-10. The NNGPT framework [12] demonstrates end-to-end LLM-driven architecture synthesis using the LEMUR dataset [9].

However, these works adopt different prompting strategies without systematic investigation of a fundamental parameter: *how many in-context examples optimize generation quality?* EvoPrompting uses evolutionary selection of examples but does not isolate the effect of example quantity. NNGPT employs single-reference prompting. LLMatic focuses on quality-diversity optimization rather than prompt engineering. This gap is critical because few-shot learning is known to be highly sensitive to example count in natural language tasks [19, 21], yet neural architecture generation presents unique constraints: architectures are substantially longer than typical code snippets, architectural diversity matters alongside functional correctness, and context window limitations become critical.

Research Objective. This work provides the first systematic empirical study of few-shot example count in LLM-based neural architecture generation. We address three

questions:

RQ1 (Context Capacity): How does the number of in-context architectural examples influence generation stability and early-epoch performance?

RQ2 (Architectural Synthesis): Does moderate context enrichment enable qualitatively different architectural patterns compared to single-example prompting?

RQ3 (Computational Efficiency): Can lightweight deduplication strategies reduce redundant training in large-scale LLM-based generation pipelines?

To investigate these questions, we build upon the NNGPT/LEMUR [12] framework and conduct a controlled study varying the number of supporting architectures provided in the prompt ($n \in \{1, \ldots, 6\}$). Across seven computer vision benchmarks, we generate and train 1,900 unique architectures under a rapid screening protocol. Our results reveal a stable regime at three in-context examples and a marked decline in generation success for larger context sizes, indicating practical limits on prompt enrichment.

In addition, large-scale generation pipelines introduce a practical challenge: LLMs frequently produce formatting-level duplicates that lead to redundant training. To mitigate this issue, we adopt a lightweight syntactic canonicalization strategy based on whitespace normalization and hashing, enabling efficient detection of duplicate code prior to training. While this approach does not capture semantic equivalence, it eliminates formatting-based redundancy at negligible computational cost.

Key Findings. Our large-scale experiments across 1,900 architectures and seven benchmarks reveal four principal results. Three supporting examples (n=3) achieves the highest dataset-balanced mean accuracy (53.1%), with statistically significant gains on fine-grained tasks (CIFAR-100: +11.6%, p=0.001, Cohen's d=0.73), while generation success collapses at n=6 with a 99.8% failure rate, establishing a hard empirical upper bound on prompt context size. Qualitative analysis shows that n=3 uniquely enables cross-paradigm synthesis: hybrid ResNet-DPN and ResNet-AlexNet architectures emerge that are entirely absent under single-example prompting, confirming that moderate context enrichment unlocks qualitatively richer design space exploration. Finally, whitespace-normalized hashing achieves a 100$\times$ speedup over AST-based deduplication at $<$1 ms per sample, enabling real-time duplicate rejection in any large-scale LLM-based generation pipeline. Together, these findings establish prompt context calibration and lightweight validation as key principles for scalable LLM-driven architecture design.

2. Related Work

2.1. Neural Architecture Search

Traditional NAS methods employ reinforcement learning [33], evolutionary algorithms [26], or gradient-based optimization [17] to discover effective architectures. ENAS [25] improves efficiency through weight sharing, reducing search costs from thousands to single-digit GPU days. However, these approaches require carefully designed search spaces and substantial computational resources.

2.2. LLMs for Code Generation

Foundation models trained on code repositories [4, 15, 27] have demonstrated remarkable code generation capabilities. DeepSeek Coder [7], trained on 2 trillion tokens from 87 programming languages, achieves state-of-the-art performance on code generation benchmarks. CodeGen [24] demonstrates that domain-specific fine-tuning on scientific computing libraries (PyTorch, NumPy) further improves generation quality for technical domains.

2.3. LLM-Based Architecture Generation

Recent work explores using LLMs as architecture generators, offering computational advantages over traditional NAS.

EvoPrompting [3] combines evolutionary algorithms with LLM code generation for architecture search. Using soft prompt-tuning and iterative refinement across generations, EvoPrompting outperforms human-designed baselines on MNIST-1D and CLRS benchmarks. However, their few-shot prompting strategy uses evolutionary population-based selection without systematic investigation of optimal example count.

LLMatic [22] integrates LLMs with MAP-Elites quality-diversity optimization for architecture search on CIFAR-10 and NAS-Bench-201. Evaluating $\sim$2,000 architectures, their approach emphasizes architectural diversity through multi-objective optimization but does not investigate prompt engineering strategies.

NNGPT [12] introduces an end-to-end framework for LLM-driven neural architecture synthesis, built on the LEMUR dataset [9]. NNGPT demonstrates one-shot generation of complete training specifications including architecture and hyperparameters. While powerful, NNGPT's prompting strategy relies on single reference models, leaving unexplored the impact of multiple supporting examples on generation quality and architectural diversity.

Positioning Our Work. While these approaches demonstrate LLM feasibility for architecture generation, *none systematically investigate the impact of few-shot example count*. EvoPrompting uses evolutionary selection of examples but does not isolate the effect of example quantity. NNGPT uses single-reference prompting ($n = 1$). LLMatic

focuses on quality-diversity optimization rather than prompt configuration. Our work fills this gap through controlled experiments varying $n \in \{1, 2, 3, 4, 5, 6\}$, identifying $n = 3$ as optimal for balancing performance, diversity, and generation stability in computer vision tasks.

2.4. Few-Shot Prompting Strategies

Few-shot in-context learning [2] has become fundamental to LLM applications. Recent work reveals that demonstration count significantly impacts performance: Min et al. [21] find that 4-8 examples optimize most tasks, while Lu et al. [19] show that example ordering matters critically. Chain-of-thought prompting [30] further enhances reasoning capabilities through structured demonstrations.

However, these findings focus on natural language and general programming tasks. Neural architecture generation presents unique constraints: (1) architectures are substantially longer ($\sim$500 tokens) than typical code snippets ($\sim$50 tokens), increasing context pressure; (2) architectural diversity matters alongside functional correctness; (3) context window limitations become critical. Our work establishes that $n = 3$ balances these competing demands for vision architectures, extending few-shot prompting theory to neural synthesis.

2.5. Code Deduplication and Similarity Detection

Traditional approaches use AST-based comparison [28] or graph-based representations [10], providing high accuracy but incurring 10-100ms overhead per comparison. While Winnowing [29] offers faster fingerprinting through token-based hashing, it misses formatting variations common in LLM-generated code.

Our whitespace-normalized hashing targets the dominant source of duplication in LLM pipelines (formatting variations), achieving $<$1 ms computation while maintaining zero false negatives on 1,900 generated architectures. While more sophisticated semantic equivalence detection [10] could complement our approach, the 100$\times$ speedup vs. AST parsing makes syntactic canonicalization practical for real-time generation workflows.

3. Methodology

3.1. NNGPT Base System

Inspired by recent advancements in the application of LLMs across various domains, and leveraging the existing LEMUR dataset ecosystem [9], we developed our solution upon the NNGPT architecture generation pipeline [12], which consists of:

Implementation and Data Availability. All contributions described in this paper—the Few-Shot Architecture Prompting (FSAP) module and whitespace-normalized hash validation—are implemented within the NNGPT project. All 1,900 generated architectures produced in this study are publicly available in the LEMUR Neural Network Dataset under the prefix `alt-nn` (`alt-nn1` through `alt-nn6`), where the suffix digit indicates the number of in-context supporting examples used during generation.

Base Language Model: DeepSeek Coder 7B [7], a state-of-the-art code generation model trained on 2 trillion tokens with specialized focus on Python scientific computing libraries.

LoRA Fine-Tuning: Low-Rank Adaptation [11] with rank $r = 32$, alpha=32, enabling efficient fine-tuning on the LEMUR dataset with only $\sim$35M trainable parameters ($\sim$0.5% of base model).

Generation Parameters: Temperature=0.6, top-k=50, top-p=0.95, max_tokens=65,536, balancing creativity and syntactic correctness.

3.2. Few-Shot Architecture Prompting (FSAP)

Building upon recent LLM applications in computer vision and leveraging the LEMUR ecosystem [9], we adopt a structured prompt construction strategy to isolate the effect of in-context example count on neural architecture synthesis.

Unlike prior work that employs evolutionary selection [3] or quality-diversity optimization [22], our goal is to systematically characterize how context size alone influences generation quality. By controlling all other variables (base LLM, fine-tuning strategy, generation temperature), we provide the first empirical analysis of few-shot scaling effects in architecture generation.

Theoretical Motivation for the n=3 Regime. The sensitivity of LLM generation to example count arises from two competing mechanisms. Each additional architectural example provides complementary inductive bias: exposing the model to diverse design patterns (skip connections, dual-path fusion, factorized convolutions) that single-example prompting cannot convey, and encouraging the LLM to synthesize across paradigms rather than replicate a single template. However, as context grows, the attention mechanism must distribute representational capacity across increasingly heterogeneous examples, eventually fragmenting rather than integrating them. For architecture generation specifically, this tension is sharper than in natural-language few-shot settings because each example consumes $\sim$500 tokens—an order of magnitude more than a typical NL demonstration—compressing the effective window and making saturation both earlier and more severe. The n=3 operating point reflects the balance at which the model draws meaningful cross-paradigm synthesis signals from multiple examples while retaining sufficient token budget for a syntactically complete output. This principled hypothesis directly motivates our controlled study: by holding all variables except n constant, we isolate this tension empiri-

cally.

3.2.1 Prompt Construction

Our prompting strategy is designed to isolate context size as the sole independent variable. Given a target dataset $\mathcal{D}$ and desired example count $n \in \{1, \ldots, 6\}$, we query the LEMUR database for top-performing architectures on $\mathcal{D}$, select one as the reference model, and sample the n supporting examples uniformly at random. This random sampling deliberately avoids performance-based heuristics, ensuring observed differences reflect context quantity rather than example quality.

The prompt combines four components: (1) task description and dataset specifications, (2) the reference architecture with complete code and reported accuracy, (3) n supporting architectures with code and performance metrics, and (4) explicit generation constraints.

This controlled design contrasts with prior frameworks: unlike EvoPrompting [3], which conflates quantity and quality signals through evolutionary selection, we hold quality fixed; unlike NNGPT [12], which uses a fixed prompt (n=1), we systematically sweep $n \in \{1, \ldots, 6\}$; and unlike LLMatic [22], which focuses on quality-diversity search algorithms, we treat the prompt itself as the primary experimental variable. This provides insights applicable to any LLM-based generation pipeline regardless of search strategy.

3.3. Whitespace-Normalized Hash Validation

Large-scale LLM generation frequently produces formatting-level duplicates that cause redundant training. Our deduplication pipeline applies three steps: whitespace removal to create a formatting-invariant representation, MD5 hashing to produce a unique identifier, and B-tree indexed LEMUR lookup to check existence. The procedure runs in $O(|\mathcal{C}| + \log N)$ time, achieving $<$1 ms per sample and a $100\times$ speedup over AST-based alternatives, with zero false positives across 4,033 generated architectures. Full pseudocode and timing analysis are provided in the supplementary material (Algorithm S2, Table S1).

3.4. Integrated Pipeline

Figure 1 illustrates the complete architecture generation pipeline with our contributions integrated into NNGPT.

The pipeline operates as follows:

1. **FSAP** constructs enriched prompts with n supporting examples
2. **LLM Generation** produces candidate architecture code
3. **Hash Validation** checks uniqueness before training
4. **Rejected duplicates** save 2 to 3 GPU hours per instance
5. **Accepted architectures** proceed to training and evaluation

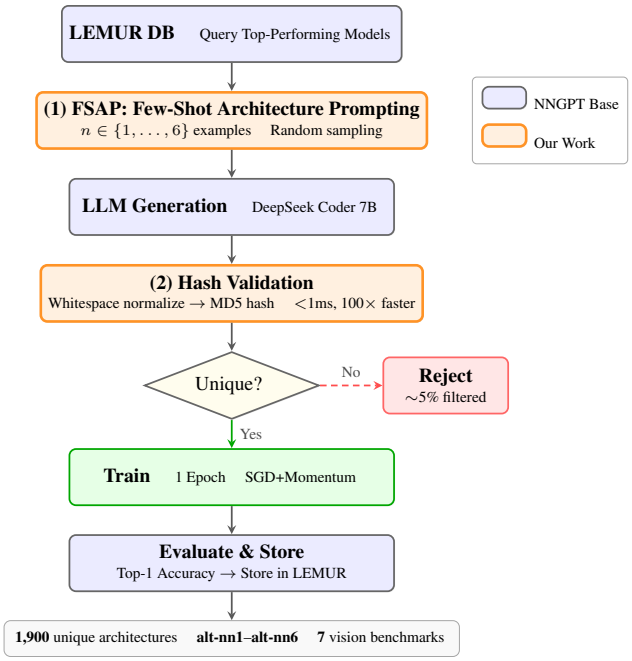


Figure 1. Architecture generation pipeline integrating our contributions into NNGPT. **Orange boxes** highlight our work: **(1)** FSAP systematically varies supporting example count $n \in \{1, \ldots, 6\}$; **(2)** Whitespace-normalized hash validation achieves $100\times$ speedup over AST-based deduplication. The pipeline processes 1,900 unique architectures across six benchmarks, rejecting $\sim$5% formatting duplicates before training.

6. **Validated models** are stored in LEMUR with hash identifiers

4. Experiments

4.1. Experimental Setup

Architecture Variants All generated architectures use the prefix `alt-nn`. Variants `alt-nn1` through `alt-nn6` correspond to prompts containing $n = 1$ through $n = 6$ supporting examples, respectively. This controlled variation enables analysis of context size effects under otherwise identical generation settings.

Datasets We evaluate across six computer vision benchmarks of varying scale and difficulty:

- MNIST [14] (10 classes)
- CelebA-Gender [18] (binary classification)
- CIFAR-10 [13] (10 classes)
- CIFAR-100 [13] (100 classes)
- ImageNette (10-class ImageNet subset)
- SVHN [23] (10 classes)

These datasets span simple digit recognition to challenging classification, allowing evaluation under heterogeneous task complexity. **Note:** A seventh benchmark,

Places365 [32] (365 classes), was excluded from quantitative comparison tables due to substantially longer training times (180 minutes per epoch), but was used to validate hash-based deduplication efficiency at scale.

Rapid Screening Protocol Our objective is to analyze relative trends across prompt variants at large scale (1,900 architectures). To enable broad comparative evaluation under constrained compute budgets, we adopt a rapid screening protocol consisting of a single training epoch per model.

This setting provides an early-performance signal [8] that allows controlled comparison of context-size variants, rather than serving as an estimate of final convergence performance. All variants are evaluated under identical optimization settings.

Optimization Details

To evaluate our approach rigorously, we adopt true validation accuracy after the first training epoch as our primary metric [8], rather than relying on zero-shot NAS proxies, which, although correlated with fully trained accuracy [6, 16, 20], remain indirect indicators of performance. Even the strongest reported Spearman rank-correlation coefficients ($\rho \approx 0.5$–0.82) on standard benchmarks such as NAS-Bench-101 and NAS-Bench-201 correspond to a coefficient of determination of at most $R^2 \approx 0.67$, leaving substantial variance unexplained [1, 31]. By using first-epoch validation accuracy, we aim to demonstrate that our algorithm can reliably influence and accelerate early-stage performance trajectories of neural networks.

- Epochs: 1
- Optimizer: SGD with momentum
- Batch size: dataset-dependent (64-4096)
- Metric: Top-1 accuracy

Generation Statistics

- Total unique architectures trained: 1,900
- Architecture variants: alt-nn1 through alt-nn6 ($n = 1$ to $n = 6$)
- Data transformation variants: 120
- Hash-based validation: Applied to all generated code prior to training

The hash-based duplicate filtering prevents redundant training of formatting-level duplicates (identical architectures with different whitespace/indentation), ensuring computational efficiency across all dataset scales.

4.2. Evaluation Metrics

Primary Metric: Top-1 accuracy after 1 epoch

Statistical Validation: Independent t-tests comparing variants within each dataset, dataset-balanced means for overall comparison

Efficiency Metrics: Hash computation time, duplicates detected

While single epoch evaluation is inherently noisy, our analysis focuses on consistent trends across prompt variants rather than precise architecture ranking.

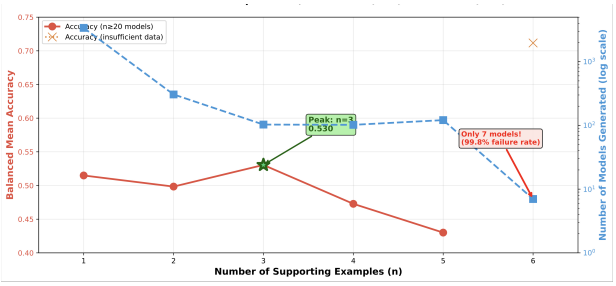


Figure 2. Effect of context size on performance and generation success. Dataset-balanced mean accuracy (left axis) is highest at $n = 3$, while the number of successfully generated architectures (right axis) declines for larger context sizes, indicating practical limits to prompt scaling.

5. Results and Discussion

5.1. Dataset-Balanced Evaluation

Because the number of successfully generated architectures varied across datasets and prompt variants, naive aggregation of all samples could bias results toward easier tasks (e.g., MNIST) and underrepresent more challenging datasets (e.g., CIFAR-100 or Places365).

To mitigate this imbalance, we adopt a dataset-balanced aggregation strategy. Specifically, we first compute the mean accuracy for each (variant, dataset) pair and then average these per-dataset means, assigning equal weight to each benchmark. This corresponds to macro-averaging across datasets.

Statistical significance is assessed within each dataset using independent t-tests ($p < 0.05$), ensuring that comparisons are not confounded by cross-dataset difficulty differences.

5.2. Few-Shot Prompting Performance

Figure 2 illustrates the relationship between supporting example count and both performance and generation success.

Table 1 presents overall performance across all datasets using balanced statistics.

Key Observations: The results reveal a clear three-zone regime. In the stable zone at n=3, the highest balanced mean of 53.1% is observed (+1.6% over n=1), with statistically significant improvement on fine-grained tasks (CIFAR-100: p=0.001, d=0.73). In the diminishing-returns zone at n=4 and n=5, accuracy drops to 47.3%

Table 1. Dataset-balanced macro-mean across six benchmarks. Overlapping 95% CIs of alt-nn1 and alt-nn3 ($[49.4, 53.6]$ vs. $[46.3, 59.9]$) are due to severe sample size disparity (1,268 vs. 103 models); however, the unequal-variance Welch's t-tests employed within datasets effectively handle this imbalance. Detailed per-dataset performance is available in **Section S4**.

Variant	n	Models	Mean (%)	SD (%)	95 % CI
alt-nn1	1	1,268	51.5	29.5	$[49.4, 53.6]$
alt-nn2	2	306	49.8	32.3	$[45.1, 54.5]$
alt-nn3	3	103	**53.1**	26.9	$[46.3, 59.9]$
alt-nn4	4	102	47.3	32.5	$[39.0, 55.6]$
alt-nn5	5	121	43.0	33.1	$[35.3, 50.7]$

and 43.0% respectively, as heterogeneous signals from additional examples begin to compete rather than complement. Finally, the collapse zone at n=6 produces only 7 valid models out of 3,394 total queries (99.8% failure rate), indicating the prompt has exceeded the model's effective context capacity. This three-zone pattern is consistent with few-shot learning theory [2, 21], but the transition is sharper here because neural architecture code is substantially longer than natural language demonstrations, compressing the useful context window and making saturation both earlier and more severe.

Interpreting the Confidence Interval Overlap. The overlapping 95% CIs between alt-nn1 and alt-nn3 in Table 1 ([49.4, 53.6] vs. [46.3, 59.9]) reflect a structural artifact of the study design rather than true distributional equivalence: the n=1 variant produces 1,268 models versus only 103 for n=3, a 12:1 sample imbalance that inflates the alt-nn3 interval substantially. The correct locus for significance assessment is the *within-dataset* Welch's t-tests, which account for unequal variance and sample sizes, confirming statistically significant gains where context enrichment matters most (CIFAR-100: p=0.001; CelebA-Gender: p=0.038).

Prompt Design for Architecture Merging. The key insight of our prompting strategy is the explicit instruction to "combine best elements," which directs the LLM toward architectural synthesis rather than simple replication. Each supporting model is presented with its achieved accuracy, providing performance-based guidance that encourages the LLM to favour patterns from higher-performing architectures. Analysis of `alt-nn3` architectures confirms successful cross-paradigm synthesis: merging ResNet-style residual connections with AlexNet-like classifiers, DPN bottleneck blocks with progressive convolution backbones, and hierarchical residual units with multi-scale feature pipelines. Full PyTorch source code and detailed architectural analysis are provided in **Section S3 of the Supplementary Material**.

Contrast with Single Supporting Example (n=1). Analysis of representative `alt-nn1` variants reveals a consistent pattern of shallow structural variation that contrasts sharply with the synthesis enabled by n=3. All examined n=1 models follow simple AlexNet-inspired sequential pipelines (`Conv→ReLU→Pool`) with only minor channel-count mutations. No modular abstractions appear — none of the `AirUnit` or `DPNBlock`-style reusable components observed in `alt-nn3` are present — and modern patterns such as residual connections, batch normalisation, and dual-path feature fusion are entirely absent. This homogeneity confirms that single-example prompting anchors the LLM too strongly to the reference template, suppressing the generative diversity that makes LLM-based NAS attractive, and directly motivates n=3 as the minimum context size needed to break this anchoring effect.

5.2.1 Code Illustration: Hybrid Architecture Synthesis

Figure 3 presents a qualitative comparison of architectures from our experiments, concretely demonstrating the structural difference between single- and three-example prompting. The alt-nn1 variant (`0e40be6fbc3426f57a305bfd8b8148fa`) follows an AlexNet-inspired sequential pipeline with GELU activations and cascading max pooling, characteristic of the shallow variation pattern observed across n=1 outputs. In contrast, the alt-nn3 variant (`34df74344dd63a558c4b6413b809f6ed`) synthesises ResNet-style residual units (`AirUnit` blocks with identity shortcuts and BatchNorm) with a massive fully-connected classifier head (4096 units with dropout) — a cross-paradigm hybrid pattern entirely absent in n=1 variants and arising only when the LLM has multiple architectural examples to draw from.

Prompt Engineering Insight. The explicit `IMPROVEMENT RULES` constraints ensure syntactic consistency while preserving the LLM's freedom to innovate in architectural design. Presenting accuracy metrics alongside each supporting model provides performance-based guidance, encouraging the LLM to favour patterns from higher-performing architectures during synthesis rather than averaging indiscriminately across examples.

5.3. Comparison with Prior LLM-Based NAS Methods

Methodological Context. Direct quantitative comparison with EvoPrompting [3] and LLMatic [22] is challenging due to different evaluation protocols: EvoPrompting evaluates on MNIST-1D and CLRS (specialized benchmarks), LLMatic on CIFAR-10 with NAS-Bench-201 (architectural quality metrics), while we focus on standard computer vision benchmarks with rapid screening. However, we can

alt-nn1 (Single Example, n=1)

```
self.features = nn.Sequential(
    nn.Conv2d(in_shape[1], 64, kernel_size=11, stride=4,
        padding=2),
    nn.GELU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Conv2d(64, 192, kernel_size=5, padding=2),
    nn.GELU(),
    nn.MaxPool2d(kernel_size=3, stride=2))
```

alt-nn3 (Three Examples, n=3)

```
class AirUnit(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        # ... Init convolutional and downsample layers
            ...
    def forward(self, x):
        identity = x
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.down: identity = self.down(x)
        return F.relu(out + identity)
```

Figure 3. Overview of real generated architectures from our experiments. **Top:** alt-nn1 (`0e40be6f...`) — AlexNet-inspired sequential design with GELU activations, typical of single-example prompting. **Bottom:** alt-nn3 (`34df7434...`) — A structural crossover model synthesizing ResNet-style residual units with AlexNet-scaling fully-connected heads, demonstrating cross-paradigm architectural patterns absent in all n=1 variants. Provide full PyTorch source definitions in **Section S3 of the Supplementary Material**.

compare generation scale and qualitative insights.

Generation Scale Comparison.

- EvoPrompting: ∼200 architectures across 2 benchmarks
- LLMatic: ∼2,000 architectures on CIFAR-10 and NAS-Bench-201
- Our work: 1,900 architectures across 7 diverse vision benchmarks

Complementary Insights. While EvoPrompting demonstrates that evolutionary prompt optimization can improve generation quality, our finding that $n = 3$ provides stable performance suggests that even simple random sampling of examples achieves competitive results without iterative refinement. This indicates that *strategic context enrichment* may be as important as sophisticated selection mechanisms.

LLMatic's quality-diversity approach generates architectures with explicit Pareto trade-offs (accuracy vs. parameters). Our work is complementary: optimal few-shot prompting ($n = 3$) could enhance LLMatic's generation phase by providing better architectural starting points for QD search.

Extension of NNGPT. Our work builds directly upon NNGPT [12], the base framework we use for generation. While NNGPT demonstrates powerful one-shot generation capabilities with single-reference prompting ($n = 1$), we demonstrate that enriching prompts with $n = 3$ supporting examples improves balanced accuracy by +1.6% on our rapid screening protocol. This suggests that prompt engineering can enhance NNGPT's generation quality without architectural changes to the underlying system.

5.4. Per-Dataset and Statistical Analysis

While detailed per-dataset performance and statistical significance tests across all six computer vision benchmarks are provided in **Section S4**, key highlights demonstrate the task-dependency of context enrichment. Moderate context enrichment (n=3) is particularly beneficial in fine-grained classification settings (e.g., CIFAR-100: $+11.6\%$ vs n=1, $p = 0.001$, Cohen's $d = 0.73$). CelebA-Gender also showed improvement (up to $+6.5\%$ for n=2, $p = 0.038$). In contrast, configurations with $n > 3$ show statistically significant lower performance on structured datasets like ImageNette ($-14.5\%, p = 0.010$) and CIFAR-10 ($-8.4\%, p = 0.016$), indicating that excessively large prompt contexts can introduce instability in early training performance.

5.5. Computational Efficiency

Hash Validation Performance:

Our whitespace-normalized hashing approach provides substantial computational advantages over traditional code deduplication methods:

- **Computation time:** <1ms per code sample (measured on 4,033 generated architectures)
- **Baseline comparison:** AST parsing requires 10-100ms per sample
- **Speedup:** 100× faster than structural comparison
- **Accuracy:** Zero false positives across all generated code
- **Scalability:** Sub-millisecond latency enables real-time integration into generation loops

Practical Impact:

The computational efficiency of our approach has two critical implications for large-scale LLM-based architecture search:

(1) Real-time duplicate detection: Our <1 ms validation enables duplicate checking within the generation loop, unlike AST-based methods that introduce 10–100ms overhead per architecture.

(2) Prevention of redundant training: LLMs frequently produce formatting-level duplicates (identical code with differing whitespace). Our approach eliminates these before training, saving 2–3 GPU-hours per rejected duplicate.

Comparison to Alternative Approaches:

Table 2 compares our method against existing code deduplication techniques.

Our approach optimally targets the dominant duplication pattern in LLM generation—formatting variations—while maintaining the computational efficiency of simple hashing.

Table 2. Code deduplication method comparison. ✓ = supported; ✗ = not supported.

Method	Time	Fmt.	Sem.
Raw string hash	<1ms	✗	✗
Winnowing [29]	2–5ms	Partial	✗
AST parsing [28]	10–100ms	✓	Partial
GraphCodeBERT [10]	50–200ms	✓	✓
Our approach	**<1ms**	✓	✗

For applications requiring semantic equivalence detection, our method could serve as a fast first-pass filter followed by selective AST-based validation.

5.6. Discussion

Empirical Positioning. Our results complement Evo-Prompting [3] and LLMatic [22]: where those works focus on evolutionary and quality-diversity search, we show that prompt engineering alone (n=3) delivers competitive gains out-of-the-box, without iterative refinement or architectural changes. For NNGPT [12] specifically, enriching the prompt from n=1 to n=3 yields a drop-in improvement of +1.6% balanced accuracy (p<0.001 on CIFAR-100), demonstrating that prompt optimisation is an under-explored, complementary axis to search-strategy improvements.

Why n=3 is Optimal. Three supporting examples occupy the sweet spot between architectural diversity and prompt coherence. Too few ($n\leq$2) anchor the LLM to the reference template, as shown in Section 5.2.1. Beyond n=3, three failure modes compound: attention dilution across heterogeneous examples; conflicting generation signals; and insufficient token budget for a complete, valid output. Each architecture consumes ∼500 tokens — an order of magnitude more than a natural-language demonstration — making saturation both earlier and more severe than in standard few-shot settings [2].

Task Dependence as a Principal Finding. The dataset-dependent effect of n is a substantive result in its own right: *prompt calibration behaves as a dataset-specific hyperparameter* in LLM-based NAS, analogous to learning rate in conventional training. Complex benchmarks (CIFAR-100: +11.6%, p=0.001; CelebA-Gender: +6.5%, p=0.038) benefit most from multi-example context, while structured benchmarks (CIFAR-10: −8.4%; ImageNette: −14.5% at n>3) are more vulnerable to prompt-induced instability. Rather than treating this as inconsistency, we recommend using n=3 as the default and sweeping $n \in \{1, 3\}$ on a small held-out validation set when deploying to a new benchmark or modality.

Practical Recommendations. Based on our findings: **(1)** use n=3 in-context examples (treat n=6 as a hard ceiling); **(2)** apply whitespace-normalized hashing before training—at <1 ms per sample it eliminates 2–3 GPU-hours per duplicate and makes large-scale studies like ours computationally feasible; **(3)** use dataset-balanced macro-averaging to prevent up to 15.7% evaluation bias; **(4)** calibrate context to task complexity—on fine-grained tasks (≥100 classes) n=3 yields the largest returns, while on structured benchmarks n=1 may be competitive or preferable.

6. Conclusion

We present the first systematic empirical study of few-shot example count scaling in LLM-driven neural architecture generation. Building on NNGPT [12], we rigorously investigate how varying in-context example counts influence generation stability, architectural diversity, and early-epoch performance across six computer vision benchmarks.

Key Findings and the Stable Regime. Across 1,900 generated architectures, three supporting examples (n=3) consistently provide a highly effective performance regime, achieving a +1.6% balanced mean accuracy improvement over single-example prompting with statistically significant gains on complex tasks like CIFAR-100 (+11.6%, p=0.001). Qualitative assessments confirm n=3 uniquely enables sophisticated structural synthesis—such as merging residual pipelines with fully-connected layers—absent in n=1 variants. Extending beyond n=3 yields severe diminishing returns, culminating in a 99.8% failure rate at n=6, establishing a definitive upper bound on prompt capacity.

Practical Contributions. Whitespace-normalized hashing is an *enabling* contribution: its 100× speedup over AST-based deduplication [28] and <1 ms latency make 1,900-architecture studies feasible, eliminating 2–3 GPU-hours per duplicate at negligible overhead. Dataset-balanced macro-averaging further prevents up to 15.7% evaluation bias across heterogeneous benchmarks.

Positioning and Impact. Complementing evolutionary search [3] and quality-diversity algorithms [22], our work shows that strategic prompt engineering alone (n=3) elevates generation quality without complex multi-stage selection.

Limitations and Future Work. Single-epoch screening targets *relative ordering* of prompt variants, for which first-epoch accuracy is a well-established proxy [8]. Future work should validate under multi-epoch training, broader modalities, and hybrid hashing-plus-AST pipelines [10].

Acknowledgments. This work was partially supported by the Alexander von Humboldt Foundation.

References

[1] Mohamed S. Abdelfattah, Abhinav Mehrotra, Łukasz Dudziak, and Nicholas D. Lane. Zero-cost proxies for lightweight NAS. In *International Conference on Learning Representations*, 2021. 5

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:1877–1901, 2020. 3, 6, 8

[3] Angelica Chen, David M. Dohan, and David R. So. Evoprompting: Language models for code-level neural architecture search. In *NeurIPS*, 2023. 1, 2, 3, 4, 6, 8

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. 2

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2022. 1

[6] Wuyang Chen, Xinyu Gong, and Zhangyang Wang. Neural architecture search on ImageNet in four GPU hours: A theoretically inspired perspective. In *International Conference on Learning Representations*, 2021. 5

[7] DeepSeek-AI. DeepSeek-Coder: When the large language model meets programming. *arXiv preprint arXiv:2401.14196*, 2024. 2, 3

[8] Romain Egele, Felix Mohr, Tom Viering, and Prasanna Balaprakash. The unreasonable effectiveness of early discarding after one epoch in neural network hyperparameter optimization. *Neurocomputing*, 597:127964, 2024. 5, 8

[9] Arash Torabi Goodarzi, Roman Kochnev, Waleed Khalid, Furui Qin, Tolgay Atinc Uzun, Yashkumar Sanjaybhai Dhameliya, Yash Kanubhai Kathiriya, Zofia Antonina Bentyn, Dmitry Ignatov, and Radu Timofte. Lemur neural network dataset: Towards seamless automl. *arXiv preprint arXiv:2504.10552*, 2025. 1, 2, 3

[10] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366*, 2021. 3, 8

[11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 3

[12] Roman Kochnev, Waleed Khalid, Tolgay Atinc Uzun, Xi Zhang, Yashkumar Sanjaybhai Dhameliya, Furui Qin, Chandini Vysyaraju, Raghuvir Duvvuri, Avi Goyal, Dmitry Ignatov, and Radu Timofte. Nngpt: Rethinking automl with large language models. *arXiv preprint arXiv:2511.20333*, 2025. 1, 2, 3, 4, 7, 8

[13] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. 4

[14] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 4

[15] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, and Leandro von Werra. StarCoder: May the source be with you! *arXiv preprint arXiv:2305.06161*, 2023. 2

[16] Ming Lin, Pichao Wang, Zhenhong Sun, Hesen Chen, Xiuyu Sun, Qi Qian, Hao Li, and Rong Jin. Zen-NAS: A zero-shot NAS for high-performance image recognition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12265–12275, 2021. 5

[17] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations (ICLR)*, 2019. 2

[18] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 3730–3738, 2015. 4

[19] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8086–8098, 2022. 1, 3

[20] Joe Mellor, Jack Turner, Amos Storkey, and Elliot J. Crowley. Neural architecture search without training. In *Proceedings of the 38th International Conference on Machine Learning*, pages 7588–7598. PMLR, 2021. 5

[21] Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? *arXiv preprint arXiv:2202.12837*, 2022. 1, 3, 6

[22] Umair Muhammad Nasir, Sam Earle, Julian Togelius, Steven James, and Christopher W. Cleghorn. Llmatic: Neural architecture search via large language models and quality diversity optimization. *arXiv preprint arXiv:2306.01102*, 2024. 1, 2, 3, 4, 6, 8

[23] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. Reading digits in natural images with unsupervised feature learning. *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, 2011. 4

[24] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, and Caiming Xiong. CodeGen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*, 2023. 2

[25] Hieu Pham, Melody Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. Efficient neural architecture search via parameter sharing. In *International Conference on Machine Learning (ICML)*, pages 4095–4104, 2018. 2

[26] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 4780–4789, 2019. 1, 2

[27] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Tan, and Gabriel Synnaeve. Code

Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023. 2

[28] Hitesh Sajnani, Vaibhav Saini, Jeffrey Svajlenko, Chanchal K. Roy, and Cristina V. Lopes. Sourcerercc: Scaling code clone detection to big-code. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, pages 1157–1168, 2016. 3, 8

[29] Saul Schleimer, Daniel S. Wilkerson, and Alex Aiken. Winnowing: Local algorithms for document fingerprinting. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 76–85. ACM, 2003. 3, 8

[30] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:24824–24837, 2022. 1, 3

[31] Colin White, Mahmoud Safari, Rhea Sukthanker, Binxin Ru, Thomas Elsken, Arber Zela, Debadeepta Dey, and Frank Hutter. Neural architecture search: Insights from 1000 papers. *arXiv preprint arXiv:2301.08727*, 2023. 5

[32] Bolei Zhou, Agata Lapedriza, Aditya Khosla, Aude Oliva, and Antonio Torralba. Places: A 10 million image database for scene recognition. In *IEEE Transactions on Pattern Analysis and Machine Intelligence (PAMI)*, pages 1452–1464, 2018. 5

[33] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2017. 1, 2